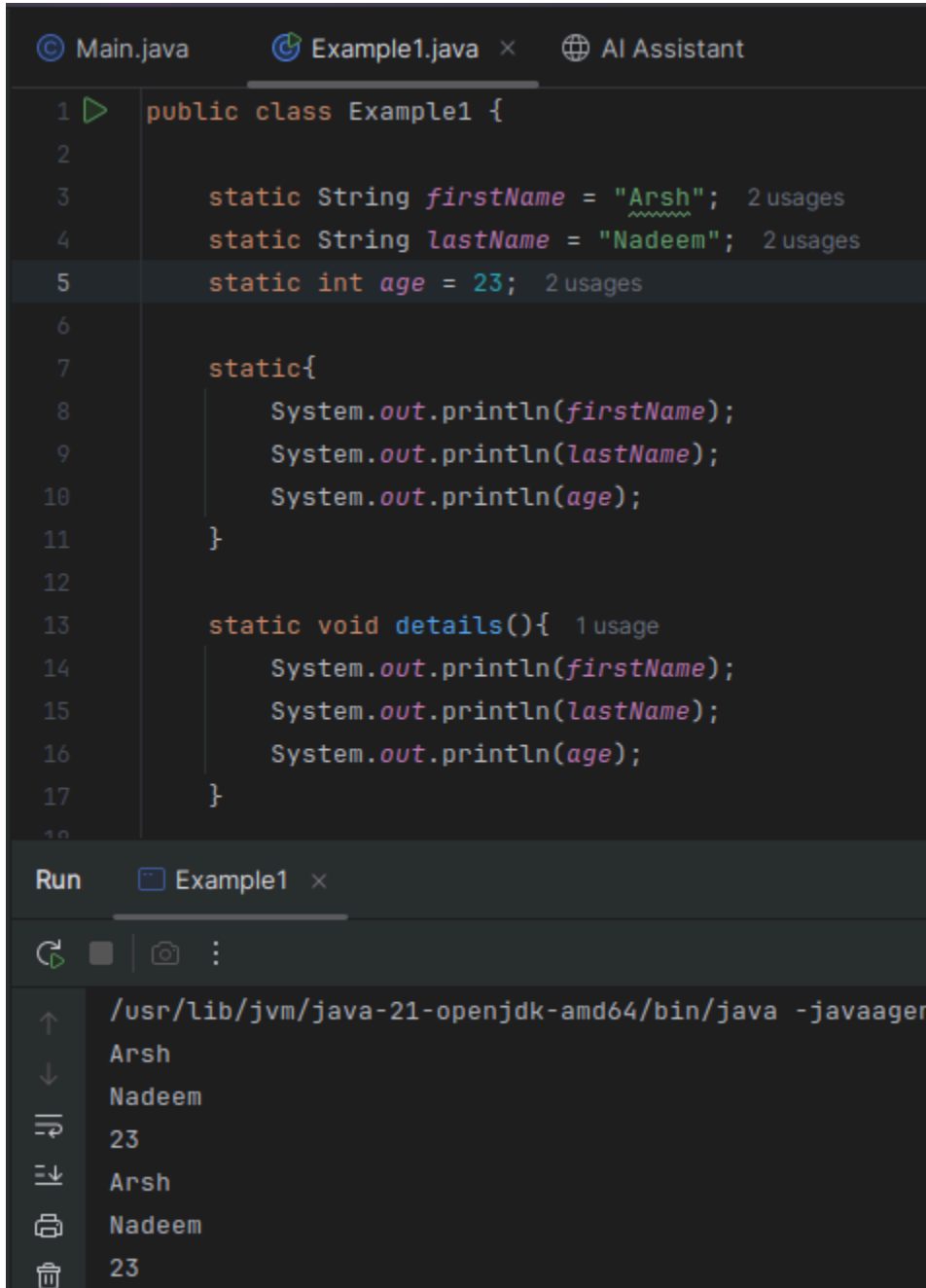


## 8218-Arsh Nadeem-Introduction to Java 1-Jan-2025

1. Write a class with FirstName, LastName & age field. Print Firstname, LastName & age using static block, static method & static variable respectively.



```
1 public class Example1 {
2
3     static String firstName = "Arsh"; 2 usages
4     static String lastName = "Nadeem"; 2 usages
5     static int age = 23; 2 usages
6
7     static{
8         System.out.println(firstName);
9         System.out.println(lastName);
10        System.out.println(age);
11    }
12
13    static void details(){ 1 usage
14        System.out.println(firstName);
15        System.out.println(lastName);
16        System.out.println(age);
17    }
18 }
```

Run Example1

/usr/lib/jvm/java-21-openjdk-amd64/bin/java -javaagent

Arsh

Nadeem

23

Arsh

Nadeem

23

2. Write a program to read user input until the user writes XDONE and then show the entered text by the user on the command line.

```
1  import java.util.Scanner;
2
3  public class SimpleInputReader {
4      public static void main(String[] args) {
5          Scanner sc = new Scanner(System.in);
6          //using string-builder to store the result
7          StringBuilder result = new StringBuilder();
8
9          System.out.println(" Enter 'XDONE' to finish");
10
11         String line;
12         //checks if line equals to XDONE.
13         while(! (line = sc.nextLine()).equals("XDONE")){
14             result.append(line).append("\n");
15         }
16
17         System.out.println("Arsh Entered:\n"+ result);
18         sc.close();
19     }
20 }
21
```

Output:

```
arsh@TTNPL-arshnadeem:src (master)$ ls
Example1.java  Main.java  SimpleInputReader.class  SimpleInputReader.java
arsh@TTNPL-arshnadeem:src (master)$ javac SimpleInputReader.java
\arsh@TTNPL-arshnadeem:src (master)$ java SimpleInputReader
Enter 'XDONE' to finish
hi arsh here
i am exiting now
XDONE
Arsh Entered:
hi arsh here
i am exiting now

arsh@TTNPL-arshnadeem:src (master)$
```

3. Write a java program to show following menu to the user:

- i. Calculate Area of Circle
- ii. Calculate Circumference of a Circle
- iii. Exit. Choose an option (1-3): Take radius as user input.

```
1  import java.util.Scanner;
2
3  public class Question3 {
4
5      static float area(float r){...}
6
7
8
9
10     static float circumference(float r){ 1 usage
11         System.out.print("Circumference is: ");
12         return 2*(22f/7)*r;
13     }
14     public static void main(String[] args) {
15         System.out.println("*****Menu*****");
16         System.out.println("1. Calculate Area of Circle \n" +
17             "2. Calculate Circumference of a Circle\n" +
18             "3. Exit. \n");
19         System.out.println("Choose an option (1-3):");
20
21         Scanner sc = new Scanner(System.in);
22         int num = sc.nextInt();
23
24         switch(num){
25             case 1:
26                 System.out.println("Enter the Radius");
27                 float radius1 = sc.nextFloat();
28                 System.out.println(area(radius1));
29                 break;
30             case 2:
31                 System.out.println("Enter the Radius");
32                 float radius2 = sc.nextFloat();
33                 System.out.println(circumference(radius2));
34                 break;
35             case 3:
36                 break;
37             default:
38                 System.out.println("invalid input");
39         }
40     }
41 }
42
```

I have put the whole code in main in a while true box to test multiple cases to get the multiple output.

```
*****Menu*****
1. Calculate Area of Circle
2. Calculate Circumference of a Circle
3. Exit.

Choose an option (1-3):
1
Enter the Radius
2
Area is: 12.571428
*****Menu*****
1. Calculate Area of Circle
2. Calculate Circumference of a Circle
3. Exit.
```

4. Create a two dimensional array of integers and display:
- sum of all elements of each column
  - sum of all elements of each row

```
import java.util.Scanner;

public class Question4 {
    public static void main(String[] args) {
        int[][] arr = new int[4][4];
        Scanner sc = new Scanner(System.in);

        System.out.println("enter the element of 4*4: ");
        for (int i = 0; i < 4; i++) {
            for (int j = 0; j < 4; j++) {
                arr[i][j] = sc.nextInt();
            }
        }

        for (int i = 0; i < 4; i++) {
            int rowSum = 0;
            for (int j = 0; j < 4; j++) {
                rowSum += arr[i][j];
            }
            System.out.println("sum of row " + i + " is: " + rowSum);
        }
    }
}
```

```
for (int i = 0; i < 4 ; i++) {
    int colSum =0;
    for (int j = 0; j < 4; j++) {
        colSum+= arr[j][i];
    }
    System.out.println( "sum of col " + i + " is: " + colSum);
}
```

Question4 x

/usr/lib/jvm/java-21-openjdk-amd64/bin/java -javaagent:/opt/intellij/lib/idea\_rt.jar=44555  
enter the element of 4\*4:

```
2 1 3 4
1 2 3 4
1 1 1 1
5 6 7 8
sum of row 0 is: 10
sum of row 1 is: 10
sum of row 2 is: 4
sum of row 3 is: 26
sum of col 0 is: 9
sum of col 1 is: 10
sum of col 2 is: 14
sum of col 3 is: 17
```

5. Create a class named Employee with fields firstname,lastname,age and designation. The class should:

- i. have all types of constructors to initialize the object
- ii. class should also have setter methods to update a particular field
- iii. Override its toString method to display a meaningful message using all these fields.

```

public class Employee {
    |
    String fName; 3 usages
    String lName; 3 usages
    int age; 3 usages
    String designation; 3 usages

    public Employee(String fName, String lName, int age, String designation) { 1 usage
        this.fName = fName;
        this.lName = lName;
        this.age = age;
        this.designation = designation;
    }

    public Employee() { no usages
        System.out.println("default constructor");
    }

    public void setFName(String fName) { no usages
        this.fName = fName;
    }

    public void setLName(String lName) { no usages
        this.lName = lName;
    }
}

```

```

public void setAge(int age) { no usages
    this.age = age;
}

public void setDesignation(String designation) { 1 usage
    this.designation = designation;
}

@Override 9 usages
public String toString() {
    return "Employee Name: " + fName + " " + lName + "\n" +
        "Employee age: "+age+"\n" + "Designation: "+designation;
}

public static void main(String[] args) {

    Employee emp = new Employee( fName: "Arsh", lName: "Nadeem", age: 23, designation: "SDE trainee");
    System.out.println(emp.toString());
    emp.setDesignation("Software developer trainee");
    System.out.println();
    System.out.println(emp.toString());
}

```

Employee Name: Arsh Nadeem

Employee age: 23

Designation: SDE trainee

Employee Name: Arsh Nadeem

Employee age: 23

Designation: Software developer trainee

6. Implement a banking system using java. Create 3 sub class of Bank : SBI, BOI, ICICI Classes should have attributes like Name, headofficeAddress, chairmanName, branchCount, fdInterestRate, personalLoanInterestRate, homeLoanInterestRate. All 3 should have following methods:

- i. add getters and setters for the fields
- ii. print details of every bank (override toString)

```
package bankingSystem;

public class Bank { 3 usages 3 inheritors

    String Name; 4 usages
    String headofficeAddress; 4 usages
    String chairmanName; 4 usages
    int branchCount; 4 usages
    float fdInterestRate; 4 usages
    float personalLoanInterestRate; 4 usages
    float homeLoanInterestRate; 4 usages

    public Bank() { System.out.println("Default Constructor"); }

    public Bank(String name, String headofficeAddress, String chairmanName, int branchCount, 3 usages
        float fdInterestRate, float personalLoanInterestRate, float homeLoanInterestRate) {
        Name = name;
        this.headofficeAddress = headofficeAddress;
        this.chairmanName = chairmanName;
        this.branchCount = branchCount;
        this.fdInterestRate = fdInterestRate;
        this.personalLoanInterestRate = personalLoanInterestRate;
        this.homeLoanInterestRate = homeLoanInterestRate;
    }

    public String getName() { no usages
        return Name;
    }

    public String getHeadofficeAddress() { no usages
        return headofficeAddress;
    }

    public String getChairmanName() { no usages
        return chairmanName;
    }
}
```

```
    public int getBranchCount() { no usages
        return branchCount;
    }

    public float getFdInterestRate() { no usages
        return fdInterestRate;
    }

    public float getPersonalLoanInterestRate() { no usages
        return personalLoanInterestRate;
    }

    public float getHomeLoanInterestRate() { no usages
        return homeLoanInterestRate;
    }

    public void setName(String name) { no usages
        Name = name;
    }

    public void setHeadofficeAddress(String headofficeAddress) { no usages
        this.headofficeAddress = headofficeAddress;
    }

    public void setChairmanName(String chairmanName) { no usages
        this.chairmanName = chairmanName;
    }

    public void setBranchCount(int branchCount) { no usages
        this.branchCount = branchCount;
    }
}
```

```

    public void setFdInterestRate(float fdInterestRate) { no usages
        this.fdInterestRate = fdInterestRate;
    }

    public void setPersonalLoanInterestRate(float personalLoanInterestRate) { no usages
        this.personalLoanInterestRate = personalLoanInterestRate;
    }

    public void setHomeLoanInterestRate(float homeLoanInterestRate) { no usages
        this.homeLoanInterestRate = homeLoanInterestRate;
    }

    @Override 22 usages
    public String toString() {
        return "Sbi{" + '\n' +
            "Name=" + Name + '\n' +
            ", headofficeAddress=" + headofficeAddress + '\n' +
            ", chairmanName=" + chairmanName + '\n' +
            ", branchCount=" + branchCount + '\n' +
            ", fdInterestRate=" + fdInterestRate + '\n' +
            ", personalLoanInterestRate=" + personalLoanInterestRate + '\n' +
            ", homeLoanInterestRate=" + homeLoanInterestRate + '\n' +
            '}';
    }
}

```

Now the subclasses with super reference to parent

```

package bankingSystem;

public class Boi extends Bank { 2 usages
    public Boi() { 1 usage
        super( name: "BOI", headofficeAddress: "Mumbai", chairmanName: "pqr", branchCount: 300,
            fdInterestRate: 5.9f, personalLoanInterestRate: 7.8f, homeLoanInterestRate: 7.5f);
    }
}

```

```

package bankingSystem;

public class Sbi extends Bank{ 2 usages
    public Sbi() { 1 usage
        super( name: "SBI", headofficeAddress: "Gurugram", chairmanName: "abc", branchCount: 400,
            fdInterestRate: 6, personalLoanInterestRate: 8.9f, homeLoanInterestRate: 6.4f);
    }
}

```

```

package bankingSystem;

public class Icici extends Bank{ 2 usages
    public Icici() { 1 usage
        // this super refer to parent class bank allowing us to get the attributed
        super( name: "ICICI", headofficeAddress: "Delhi", chairmanName: "xyz",
            branchCount: 200, fdInterestRate: 7.2f, personalLoanInterestRate: 9.4f, homeLoanInterestRate: 8.0f);
    }
}

```



7. WAP showing try, multi-catch and finally blocks.

-> So in this program I used multiple catch boxes to catch different exceptions.

```
import java.util.InputMismatchException;
import java.util.Scanner;

public class TrycatchExample {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        System.out.println("Enter numerator and denominator");

        try{
            int num = sc.nextInt();
            int deno = sc.nextInt();
            int result = num/deno;
            System.out.println(result);

            int [] arr = {1,2,3,4};
            System.out.println("Enter which index element you want: ");
            System.out.println(arr[sc.nextInt()]);
        }catch (ArithmeticException e){
            System.out.println("arithmetic exception being caught");
        }catch (ArrayIndexOutOfBoundsException e){
            System.out.println("Going out of bound");
        }catch ( InputMismatchException e){
            System.out.println("Arsh provided Wrong input ");
        }finally {
            System.out.println("This will always execute ");
        }

    }
}
```

```

/usr/lib/jvm/java-21-openjdk-amd64/bin/java -javaagent:/opt/intellij/lib/idea_rt.jar=34091
Enter numerator and denominator
10
0
arithmetic exception being caught
This will always execute
*****done*****

Enter numerator and denominator
12
3
4
Enter which index element you want:
5
Going out of bound
This will always execute
*****done*****

Enter numerator and denominator
34
num
Arsh provided Wrong input
This will always execute
*****done*****

```

8. WAP to produce NoClassDefFoundError and ClassNotFoundException exceptions.

->Creating a missing class

```

package Exceptions;

public class MissingExample { no usages
    void show(){ no usages
        System.out.println("Hi arsh is present");
    }
}

```

->in same package create main class

```
package Exceptions;

class Main{
    public static void main(String[] args) {
        MissingExample me = new MissingExample();
        me.show();
    }
}
```

Now compile both using javac and delete 'MissingExample' using rm  
Now run using java classname

```
arsh@TTNPL-arshnadeem:Exceptions (master)$ ls
Main.java  MissingExample.java
arsh@TTNPL-arshnadeem:Exceptions (master)$ javac M
Main.java      MissingExample.java
arsh@TTNPL-arshnadeem:Exceptions (master)$ javac Main.java MissingExample.java
arsh@TTNPL-arshnadeem:Exceptions (master)$ rm MissingExample.class
arsh@TTNPL-arshnadeem:Exceptions (master)$ java Main
Error: Could not find or load main class Main
Caused by: java.lang.NoClassDefFoundError: Main (wrong name: Exceptions/Main)
arsh@TTNPL-arshnadeem:Exceptions (master)$
```

->class not found

```
package Exceptions;

public class ClassNotFound {
    public static void main(String[] args) {
        try {
            // Class.forName("java.lang.StringBuffer");
            Class.forName("arsh");
        } catch (ClassNotFoundException e) {
            System.out.println("ClassNotFoundException");
        }
    }
}
```

```
/usr/lib/jvm/java-21-openjdk-amd64/bin/java -javaagent:/opt/intellij/lib/idea_r  
ClassNotFoundException  
  
Process finished with exit code 0
```

9. Create a custom exception that does not have any stack trace.

-> firstly i created a custom exception class to perform the operation  
-> then i created my own custom exception class  
-> to avoid stacktrace, i have to override a fillInStackTracemethod  
And point it to the current message.

```
package Exceptions;  
  
public class CustomException {  
    public static void main(String[] args) {  
        try{  
            throw new MyException(" Arsh Exception");  
        }catch (MyException e){  
            System.out.println("Catching" + e.getMessage());  
            e.printStackTrace();  
        }  
    }  
}  
  
class MyException extends RuntimeException{ 2 usages  
    MyException(String message){ 1 usage  
        super(message);  
    }  
    @Override no usages  
    public synchronized Throwable fillInStackTrace() {  
        return this;  
    }  
}
```

Output:

```
Catching Arsh Exception  
Exceptions.MyException:  Arsh Exception  
  
Process finished with exit code 0
```

Output when I don't override 'fillInStackTrace()':

```
Catching Arsh Exception  
Exceptions.MyException Create breakpoint :  Arsh Exception  
    at Exceptions.CustomException.main(CustomException.java:7)  
  
Process finished with exit code 0
```